

Contents

Instructor Notes - TestAbility Workshop	1
Logistics	1
Demo of the apps	2
Demo Z-type	2
Demo Zombie CLI Game	2
JavaScript Mouse Clicking Game - protect the square	3
Search Engine	3
Any App	3
How can we investigate Testability?	4
What does it mean to investigate Testability?	4
Mob Exercise: What presuppositions exist?	4
Technology Audit	4
Application Test Ability Audit	6
Information Available	6
Features	7
Command line help	7
Feature Summary	7
Mob Worked Example - Zombie CLI	8
Terminal App automation?	8
What tools do we know can help automate?	8
Could the Operating System help?	8
What else? Embedded Jar	9
What else?	9
How might I automate CLI App?	9
Mob Worked Example - ZType	10
ZType Automation	10
Example JavaScript Automated Execution Crude Z-Type Bot	10
Good Z-Type Bot	10
Demo some of the protect the square automated hacking	11

Instructor Notes - TestAbility Workshop

Logistics

- 8 hours 9 - 5 (2 hours breaks)
- 09:00 - 10:00 (60) Introduction
 - Welcome and Intro (20)
 - What is testability (30)
 - What is auditing (10)

- 10:00 - 10:30 Technology Test Ability
 - Start Technology Test Ability Audit
 - overviews, demos, audit
- 10:30 - 11:00 Break
 - coffee and morning snack
- 11:00 - 11:30 Technology Test Ability
 - finish Technology Test Ability Audit
 - debrief
 - testing
 - debrief of testing
- 11:30 - 12:30 Application Test Ability
- 12:30 - 13:30 Break
 - lunch
- 13:30 - 14:30 Automatability Audit
- 14:30 - 15:00 Self Audit - Test Ability
- 15:00 - 15:30 Break
 - coffee and afternoon snack
- 15:30 - 16:00 Self Audit - Test Mentality
- 16:00 - 16:45 (17:00) Flex For Testing
- 17:00 - 17:30 Audit of Audits - Finish

Demo of the apps

Demo Z-type

- visit <http://zty.pe/>
 - show game in action
 - questions
 - “is this testable?”
 - “could you test this?”
 - “could you automate this?”
 - “do you know what technologies this uses?”
 - “do you know how those technologies impact your testing?”
-

Demo Zombie CLI Game

- on disk

- `java -jar cli-game.jar`
 - traditional verb noun text adventure game
 - look, examine welcomemat, look, take lockpickwire, use lockpickwire, quit-game
 - questions “do you know how to test this?”
 - “this uses different technology, would we test this differently?”
-

JavaScript Mouse Clicking Game - protect the square

- link from EvilTester.com/testability
 - shoot red squares
 - mention ‘hacking this game’ adding new features
 - there are bugs in this game
 - “is this testable?”
 - “is this automatable?”
 - “do the technologies make a difference?”
-

Search Engine

- google
 - bing
 - duck duck go
 - testability - really simple notional functionality, but complicated implementations
 - could we automate this
 - do they have features that help us test them
 - do they have apis? can we automate via GUI? via HTTP calls?
-

Any App

The aim is that we can view any app with these different models.

You are not limited to these

How can we investigate Testability?

- Look at system
- Try to test it
- From experience see what supports testing and what doesn't
- See what is hard and what is easy
- See what I can manipulate and what I can't
- What technology levels can I test at?
- What techniques can I apply?
- What tools can I use?
- What are the limits of my knowledge?

What does it mean to investigate Testability?

Specifically:

Can I test this? How well can I test this? What supports me in testing this? What hinders me in testing this? What are the defaults associated with testing this? What are the issues with testing this? What are the risks with testing this?

Generically:

- Technology
- Type of App
- Domain
- Do I understand functionality

Me in relation to the application

Mob Exercise: What presuppositions exist?

- Testability is important
 - Testability is something we can investigate
 - There is more to Testability than we might at first assume
 - Investigating Testability can change how I test and is worth investigating
-

Technology Audit

- Given the CLI game
- What would we do a Technology Audit for?
- How would that help us?
- How would we audit it?

- What questions would we ask?
- Any Technology Risks?
- What demos would we need to see?
- Would we want to see the code?

Does seeing the code bias us, or not?

How can we avoid that bias?

- Different focus,
- different models,
- different traversal approach to models.

Any existing testing?

- does the testing focus on the technology or the functionality?
- what does the testing ‘cover’?
- Java
 - JRE version? used on 1.9
 - Cross platform? should be - haven’t tried
- CLI interface
 - Mobile? Don’t know
 - Tool: Terminal used? Shell used?

Source code is available

- <https://github.com/eviltester/zombie-cli-text-adventure>
- <https://github.com/eviltester/restmud-cli-game-engine>
- Java Technology
 - Need to have a JRE
 - JRE might impact app depending on version
 - Should make it cross platform
 - Do we understand Java?
 - Do we need to understand Java?
 - How would understanding Java change the applications Testability?
- CLI interface
 - this might impact portability
- Application Architecture (3 logical tiers)
 - CLI Game Engine
 - Adventure Game Engine
 - Game Rules and Data

Do our tools count as ‘technology’ or are they for automatability?

- Java can be decompiled if source isn't available. Decompilers exist and are built into IntelliJ

This application is actually dependent on other applications, and the full game engine is used in the web version and has a lot of tests associated with it.

This game has 2 engines: CLI, Game Engine, and Game Rules - what am I testing? The rules, the CLI game engine? The Main Game Engine?

Is this the easiest way to test each of these?

Insights:

- If I don't know the technology I may not know what tools to use
- If I don't understand the programming language I cannot use the source as an oracle or way of learning
- I can't test below the level I understand e.g. I can't look at HTTP traffic if I don't understand it, or know that it exists

Application Test Ability Audit

Ease of which we can test an application and the application supports our testing.

- CLI Game Functionality - accepts input, quit-game
- What hinders us for testing?
- What functionality might be useful for testing?
- What questions do we have?
- Has our technology audit helped us?

Information Available

<https://github.com/eviltester/zombie-cli-text-adventure>

Game has multiple levels, CLI wrapper.

Game Walkthrough test might reveal information - this is in a different repo.

<https://github.com/eviltester/tech-web-test-restmud-game>

in \docs

There is also a map there

The src has the game definitions

<https://github.com/eviltester/tech-web-test-restmud-game/>

- `src/main/java/uk/co/compendiumdev/restmud/gamedata`
- `/games/gamedefinitions/TechWebTest101GameDefinitionGenerator.java`

Features

What kind of command line arguments can you pass into applications?

What features might we ask the developer to create to support our testing?

-v for version

```
java -jar cli-game.jar -v
```

Command line help

```
java -jar cli-game.jar -help
- -h` for help
- -v` for version
- -verbose`
- -cols 40` - reformat output
- -txtlog logfile.txt`
```

How could these help?

Feature Summary

- CLI Args to 'unlock features for testing'
 - -h for help
 - -v for version
 - -verbose
 - -cols 40 - reformat output
 - -txtlog logfile.txt

If use -verbose -h also see

- -jsonlog jsonlog.txt` - might be more useful for automating?
- -nobackups`
- -commandslog`

Secret features only visible in the code - search for containsKey then view actual source files

- -debug`
-

Mob Worked Example - Zombie CLI

- How could I automate this?
- Is it automatable?
- Does it exhibit automatability?
- What would make it more automatable?

CLI apps are hard to automate but there are old tools like TCL and Expect that might help.

Terminal App automation?

Perhaps the terminal apps that we use have the ability to help automate with macros.

The first automation I ever did was on Green Screen apps via a Terminal Emulator called Rhumba which exposed an API that we could automate using OLE so we wrote MS Word docs which could be interpreted and called commands via the Terminal API.

What tools do we know can help automate?

WebDriver? RestAssured?

Could they help us?

Yes GUI and REST Tools could - for parts of the architecture

Although they can't automate the CLI.

The 'game' has the same backend, same data.

So we could test it 'embedded' in the CLI but we can test aspects of it via automating at GUI and REST API.

Could the Operating System help?

The game can log its commands.

```
java -jar cli-game.jar -commandslog
```

This creates a text file called `commands.txt`

Can this help us?

Unix pipes

```
cat commands.txt | java -jar cli-game.jar -txtlog -jsonlog
java -jar cli-game.jar -txtlog -jsonlog < commands.txt
```

I could compare the output files.

What else? Embedded Jar

Because it is a Jar file, I could add it to a project and call the methods from another set of tests.

e.g. <https://www.compendiumdev.co.uk/page/casestudyjavadesktop>

The CLI wrapper doesn't quite have the architecture I need to make this as risk free as I would like e.g. CliGame.java

In the CLI Game Engine it is a loop around stdin - really want that abstracted.
<https://github.com/eviltester/restmud-cli-game-engine>

What else?

Existing automated execution.

The <https://github.com/eviltester/zombie-cli-text-adventure>

The game definition has a walkthrough test

ZombieWalkthroughTest.java

This even has a DSL to make it easy to test.

Discuss DSLs and Automated Execution Abstractions

How might I automate CLI App?

- TCL & Expect?
- Unit tests at each layer - game rules, cli engine, backend engine
- shell scripts?

```
cat commands.txt | java -jar cli-game.jar -txtlog -jsonlog
java -jar cli-game.jar -txtlog -jsonlog < commands.txt
```

Mob Worked Example - ZType

- How could I automate this?
 - <http://zty.pe/>
-

ZType Automation

- WebDriver?
 - Possible, but likely too slow
- Image Recognition and OCR?
- Unit Tests?

I don't think it is open source so the 'tests' aren't available.

- Custom JavaScript code as bookmarklets

Most people don't automate like this.

Hacking JavaScript is a tremendous learning opportunity.

Show bots.

Example JavaScript Automated Execution Crude Z-Type Bot

```
var zTypeBotAlphaChars = "abcdefghijklmnopqrstuvwxyz";
var zTypeBotCharToType = 0;
ztypebot = setInterval(function() {
    ig.game.shoot(
        zTypeBotAlphaChars.charAt(zTypeBotCharToType));
    zTypeBotCharToType = (zTypeBotCharToType + 1)%26;
}, 100);
```

Remember to stop bot:

```
clearInterval(ztypebot);
```

Good Z-Type Bot

```
ztypebot = setInterval(function() {
    if (ig.game.currentTarget == null) {
        for (var t in ig.game.targets) {
```

```

        if (ig.game.targets[t].length > 0) {
            ig.game.shoot(t);
            break;
        }
    }
} else {
    while(ig.game.currentTarget != null){
        ig.game.shoot(
            ig.game.currentTarget.remainingWord.charAt(0));
    }
}
}, 2);

```

Demo some of the protect the square automated hacking